Système de Gestion de Flotte Logistique

Architecture Microservices · Docker · Kubernetes · Kafka

Spring Boot | Node.js | Keycloak | Grafana | gRPC

Master 1 GIL · Université de Rouen · 2025-2026

Encadrants : Lydia & Luc

Plan de présentation

01 **Contexte & Objectifs**
Problématique, cas d'usage logistique

02 **Architecture Technique**
5 microservices, API Gateway, Kafka

03 **Services Développés**
Véhicules, Conducteurs, Maintenance, Localisation, Événements

04 **Sécurité & Observabilité**
Keycloak, RBAC, Prometheus, Grafana

05 **Infrastructure & CI/CD**
Docker, Kubernetes, Helm, GitHub Actions

06 **Tests & Qualité**
k6 · Playwright · JUnit · Testcontainers

07 **Démonstration**
Scénarios live, métriques temps réel

01 · Contexte & Objectifs

Problématique

- ▶️ Gérer une flotte de centaines de camions en temps réel
- ▶️ Traçabilité GPS et historique des déplacements
- ▶️ Maintenance préventive et alertes automatisées
- ▶️ Coordination conducteurs / dispatching
- ▶️ Scalabilité face aux pics de charge

Tracking GPS

Streaming temps réel gRPC + TimescaleDB

Microservices

5 services indépendants et scalables

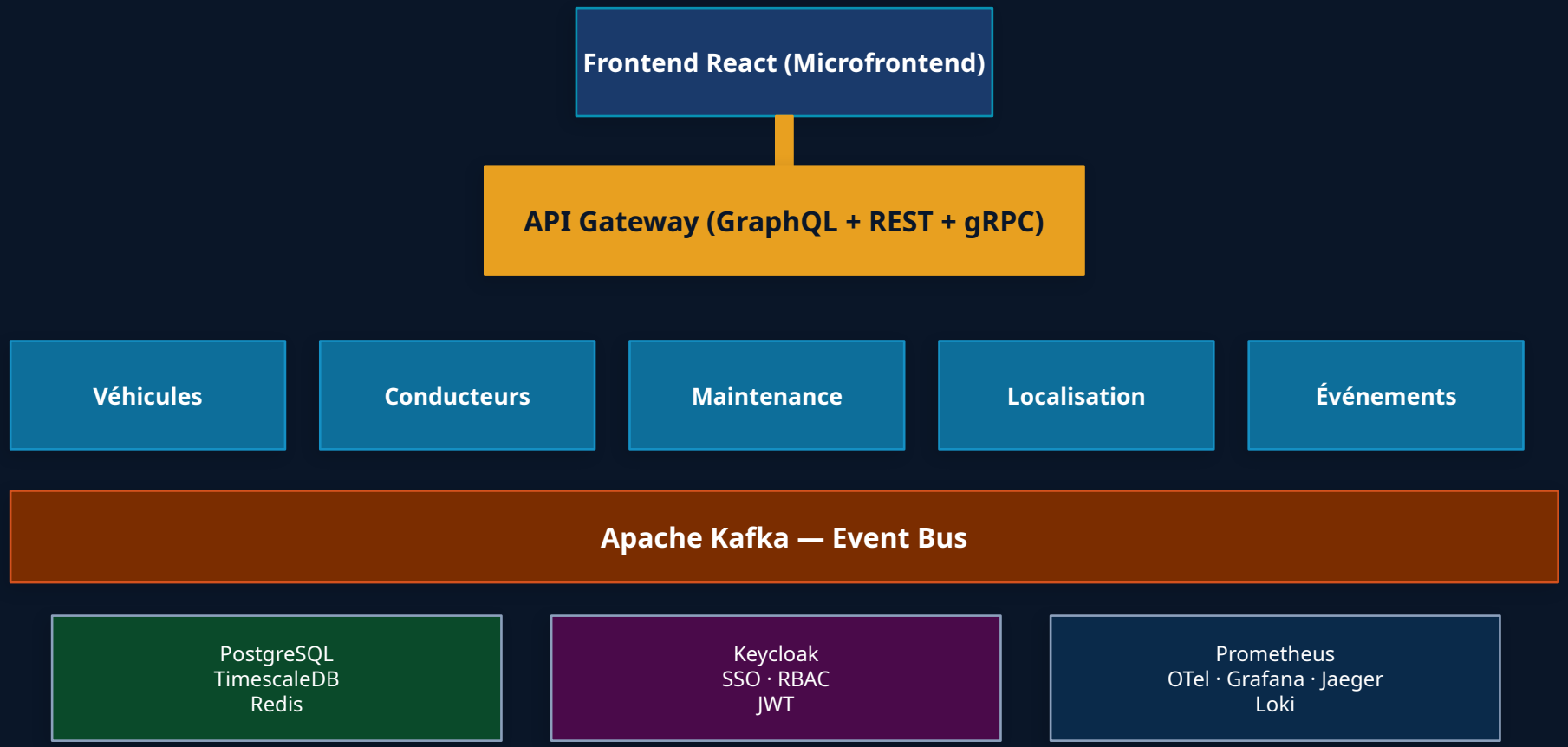
Sécurité Zero Trust

Keycloak SSO · RBAC · JWT

Observabilité

Prometheus · Grafana · Jaeger · Loki

02 · Architecture Microservices



03 · Services Développés

Véhicules

Spring Boot · Java 21

- CRUD parc automobile
- Statuts, historique, disponibilité
- Événements Kafka producer

Conducteurs

Spring Boot · Java 21

- Assignation véhicule/mission
- Profils & permis de conduire
- Validation & RBAC Keycloak

Maintenance

Spring Boot · Java 21

- Planification interventions
- Alertes préventives
- Saga pattern inter-services

Événements

Spring Boot · Java 21

- Notifications temps réel
- Agrégation des alertes multi-services
- Historique & audit des événements

Localisation

gRPC · NestJS · TimescaleDB

- Streaming GPS temps réel
- Historique time-series
- Géofencing & alertes zones

04 · Sécurité & Observabilité

🔐 Sécurité — Zero Trust

Keycloak SSO

Authentification centralisée OAuth2/OIDC

RBAC

3 rôles : Admin · Manager · Technicien

JWT

Tokens signés, validation par chaque service

mTLS

Chiffrement inter-services mutual TLS

Secrets K8s

Aucun secret en clair dans le code

📊 Observabilité — 3 Piliers

Traces

Jaeger + OpenTelemetry

Traçage distribué des requêtes

Debug inter-services visuel

Métriques

Prometheus + Grafana

RED metrics · Dashboards

Alertes seuils métier

Logs

Loki + Correlation ID

Logs JSON structurés

Recherche centralisée

05 · Infrastructure & CI/CD

Pipeline CI/CD — GitHub Actions

Code Push → Tests >80% → Docker Build → Linting & Tests → Deploy Helm K8s

Déploiement Kubernetes

Namespaces

Isolation par environnement

(dev · staging · prod)

Helm Charts

Versionning déploiements

Rollback automatique

ConfigMaps & Secrets

Configuration externalisée

Secrets chiffrés

Ingress

Routing & Load balancing

Load balancing & TLS

06 · Tests & Qualité

✅ Tests Unitaires

JUnit 5 + Mockito

Spring Boot — 80% couverture (JaCoCo)

Jest + Supertest

NestJS — 80% couverture (Istanbul)

Tests d'Intégration

Testcontainers · BDD + Kafka réels

MockMvc · Supertest

API et contrats inter-services vérifiés

Couverture globale

80% mesurée · rapport automatique CI

🌐 Tests E2E & Charge

End-to-End — Playwright

Playwright · parcours utilisateur complets

Auth Keycloak → CRUD → assertions UI

Intégré au pipeline CI (bloquant avant deploy)

Tests de Charge — k6

500 véhicules GPS en simultané

P95 < 200 ms · rapport HTML k6

RPS · erreurs · latence par endpoint

Résultats & Qualité CI

Badge couverture dans chaque PR

Résultats & Qualité CI JSON structurés

lint → test → build → deploy

Stack Technologique

Backend

Spring Boot (Java 21) · Node.js / NestJS

Base de données

PostgreSQL · Redis · TimescaleDB

Communication

REST · GraphQL · gRPC · Apache Kafka

Sécurité

Keycloak · JWT · RBAC · mTLS

Frontend

React · Vite Module Federation · Playwright E2E

Observabilité

OpenTelemetry · Jaeger · Prometheus · Grafana · Loki

Infra

Docker · Kubernetes · Helm · GitHub Actions

Bilan & Compétences acquises

5	7	80%	3
Microservices développés	Semaines de sprint	Couverture de tests	Piliers observabilité

Compétences clés démontrées :

- Architecture microservices distribuée
- Conteneurisation Docker & orchestration K8s
- Event-driven avec Apache Kafka (saga chorégraphie)
- Sécurité Zero Trust avec Keycloak
- Observabilité full-stack (traces, métriques, logs)
- Pipeline CI/CD automatisé GitHub Actions

Merci de votre attention

Questions & Démonstration